

HPC- Bubble sort

```
#include <iostream>
#include <vector>
#include <omp.h>
#include <chrono>

using namespace std;
using namespace std::chrono;

void takeUserInputArray(vector<int>& arr, int size) {
    arr.resize(size);
    cout << "Enter " << size << " elements:\n";
    for (int i = 0; i < size; i++) {
        cin >> arr[i];
    }
}

void printArray(const vector<int>& arr) {
    for (int val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

void parallelBubbleSort(vector<int>& arr, int n) {
    for (int i = 0; i < n; i++) {
        int start = i % 2;
        #pragma omp parallel for
        for (int j = start; j < n - 1; j += 2) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter the size of the array: ";
    cin >> n;

    vector<int> arr(n);

    takeUserInputArray(arr, n);

    // Measure execution time for parallel bubble sort
    auto start = high_resolution_clock::now();
    parallelBubbleSort(arr, n);
    auto stop = high_resolution_clock::now();
    auto parBubbleTime = duration_cast<milliseconds>(stop -
start).count();

    // Display the sorted array
    cout << "\nSorted array using Parallel Bubble Sort:\n";
    printArray(arr);

    // Display the execution time for parallel bubble sort
```

```
        cout << "\nParallel Bubble Sort Execution Time: " << parBubbleTime <<
" ms\n";

        return 0;
}
```

o/p:

Enter the size of the array: 6

Enter 6 elements:

1 5 4 2 3 8

Sorted array using Parallel Bubble Sort:

1 2 3 4 5 8

Parallel Bubble Sort Execution Time: 0 ms